

Exp No: 10 Date:	PERT -Analysis
-----------------------------------	-----------------------

AIM:

To perform Critical Path Method (CPM) Analysis for project scheduling and management using Python.

PROCEDURE:

Install Required Libraries: Ensure pandas, networkx, matplotlib, and numpy are installed. **Define the Problem:** Identify tasks, their durations (optimistic, most likely, pessimistic), and dependencies.

Set Up the Problem: Represent tasks as a directed graph, Calculate expected duration and variance and Perform topological sorting.

Run the Analysis: Compute earliest/latest start and finish times. Identify the critical path and project duration. Calculate standard deviation for risk assessment.

Analyze the Results: Validate the critical path and project timeline. Assess uncertainties and completion risks.

PROBLEM -1:

Activity	O	M	P	Dependencies
A	2	4	6	-
B	1	2	3	A
C	1	3	5	A
D	2	2	4	B
E	3	5	7	C
F	1	4	6	D,E

Code:

```
import numpy
as np import
networkx as nx

tasks = {
    'A': {'optimistic': 2, 'most_likely': 4, 'pessimistic': 6, 'dependencies': []},
    'B': {'optimistic': 1, 'most_likely': 2, 'pessimistic': 3, 'dependencies': ['A']},
    'C': {'optimistic': 1, 'most_likely': 3, 'pessimistic': 5, 'dependencies': ['A']},
    'D': {'optimistic': 2, 'most_likely': 2, 'pessimistic': 4, 'dependencies': ['B']},
    'E': {'optimistic': 3, 'most_likely': 5, 'pessimistic': 7, 'dependencies': ['C']},
    'F': {'optimistic': 1, 'most_likely': 4, 'pessimistic': 6, 'dependencies': ['D', 'E']}
}

# Expected time and
variance for t in tasks:
    o =
    tasks[t]['optimisti
    c'] m =
    tasks[t]['most_lik
    ely'] p =
    tasks[t]['pessimis
    tic']
    tasks[t]['te'] = (o + 4*m + p) / 6
    tasks[t]['var'] = ((p - o) / 6) **

2 G = nx.DiGraph()

    for task, details in tasks.items():
        G.add_node(task,
            duration=details['te']) for dep in
            details['dependencies']:
                G.add_edge(dep, task)
order =

list(nx.topological_sort(G)) ES,

EF = {}, {}

for node in order:
    ES[node] = max([EF.get(pred, 0) for pred in G.predecessors(node)], default=0) EF[node] =
    ES[node] + G.nodes[node]['duration']

LF, LS = {}, {}
project_duration = max(EF.values())

for node in reversed(order):
    LF[node] = min([LS.get(succ, project_duration) for succ in G.successors(node)], default=project_duration)
```

```

    LS[node] = LF[node] - G.nodes[node]['duration']
critical_path = [node for node in G.nodes if ES[node] == LS[node]] #

Project variance and std deviation

project_variance = sum(tasks[t]['var'] for t in critical_path)
project_std_dev = np.sqrt(project_variance)

print("Earliest start times:", ES)
print("Earliest finish times:", EF)
print("Latest start times:", LS)
print("Latest finish times:", LF)
print("Critical path:", critical_path)
print("Project duration (expected):", float(project_duration))

print("Project standard deviation:", project_std_dev)

```

Output:

```

Earliest start times: {'A': 0, 'B': 4.0, 'C': 4.0, 'D': 6.0, 'E': 9.0, 'F': 14.0}
Earliest finish times: {'A': 4.0, 'B': 6.0, 'C': 9.0, 'D': 8.0, 'E': 14.0, 'F': 18.0}

```

```

Latest start times: {'A': 0.0, 'B': 8.0, 'C': 4.0, 'D': 12.0, 'E': 9.0, 'F': 14.0}
Latest finish times: {'A': 4.0, 'B': 10.0, 'C': 9.0, 'D': 14.0, 'E': 14.0, 'F': 18.0}

```

```

Critical path: ['A', 'C', 'E', 'F']

```

```

Project duration (expected):

```

```

18.0

```

```

Project standard deviation: 1.2018504251546631

```

RESULT:

Thus, the Program Evaluation and Review Technique (PERT) was successfully executed and verified using Python